

Die Umstellung auf einen Blick (seit 01.06.2026)

- **AI Credits statt Premium Requests:** abgerechnet wird die tatsächliche Modell-Arbeit – tokenbasiert, zu Modell-API-Preisen.
- **1 Credit = 0,01 USD.** Budgets werden in USD gesetzt, Verbrauch in Credits angezeigt (10 \$ = 1.000 Credits).
- **Grundpreise unverändert:** Business 19 \$/Nutzer/Monat inkl. 19 \$ Credits · Enterprise 39 \$ inkl. 39 \$.
- **Kein Fallback-Modell mehr:** Verbrauch wird über Guthaben und Admin-Budgets gesteuert.

Die drei Token-Arten

Input	was Sie senden: Prompt + Kontext – wächst mit großen Dateien
Output	was Sie bekommen: Antwort + Tool-Nutzung – pro Token am teuersten
Cache	wiederverwendeter Kontext – am günstigsten, macht Sessions effizient

Was kostet was?

- **Flatrate (keine Credits):** Code Completions & Next Edit Suggestions – unbegrenzt in bezahlten Plänen.
- **Credits:** Chat · Edits · Agent Mode · Copilot CLI · Coding Agent · Spark & Spaces.
- **Credits + Actions-Minuten:** Copilot Code Review und der Coding Agent (agentische Infrastruktur).
- **Verbrauchstreiber:** Modellwahl, Prompt- & Antwortgröße, Kontext/Cache, agentische Workflows, MCP/Skills.

Pool & Budgets – so fließt das Geld

- **1. Pool zuerst:** Jede Lizenz zahlt ihr Credit-Guthaben in einen gemeinsamen Enterprise-Pool ein – alle Nutzer schöpfen zuerst daraus.
- **2. Budget = Zusatzausgaben:** Erst nach dem Pool greift das Budget – es erlaubt und begrenzt Mehrverbrauch.
- **3. Ebenen:** Enterprise → Organisation → User (universell oder individuell) – plus Cost Center für Teams.
- **4. Alerts automatisch:** Warn-Mails an Admins bei 75 / 90 / 100 %.
- **5. Budget erreicht = Stopp:** Betroffene sind blockiert bis Aufstockung oder neuer Abrechnungszyklus.
- **6. Monitoring:** Die Billing-UI trennt inkludierte und zusätzliche Ausgaben – auch mitten im Zyklus.

Faustregeln zur Modellwahl

Alltag & Fragen	Auto-Modus / Standardmodell
Boilerplate & Tests	Auto-Modus – günstig & schnell
Refactoring & Architektur	Frontier-Modell gezielt
Agent-Tasks	Frontier + /plan
Massen-Tasks	Mini-/Flash-Klasse

Kontrolle in der CLI

/usage	Verbrauch der laufenden Session
/model	aktives Modell & Angebot
/compact	Verlauf kürzen – spart Tokens

Das Prompt-Rezept

Ziel in einem Satz + drei Randbedingungen + gewünschtes Format. Iterieren ist der normale Arbeitsmodus – kein Scheitern.

1 · Klare Instruktionen

Persona vorgeben („agiere als Senior-Java-Engineer“), Format & Länge nennen, Prompt-Teile mit Trennern strukturieren, Schritte auflisten.

2 · Referenzen mitgeben

Quelltext, Doku oder Spezifikation anhängen und darauf verweisen – senkt selbstbewusst-falsche Antworten deutlich.

3 · Komplexes zerlegen

Große Aufgaben haben überproportional hohe Fehlerraten. Kleine, prüfbare Teilschritte beauftragen; lange Dialoge zusammenfassen.

4 · Denkzeit geben

Erst den Lösungsweg erklären lassen, dann den Code – das Modell „reflektiert“ und konvergiert zu besseren Antworten.

5 · Externe Tools nutzen

Tests, Linter und Compiler prüfen zuverlässiger als das Modell selbst – „... und führ mvn -q test aus“ gehört in den Auftrag.

6 · Systematisch testen

Prompt-Änderungen wie Code-Änderungen behandeln: vergleichen, messen, behalten was wirkt.

Die drei Merksätze

- Copilot ist **Assistent, nicht Autopilot** – jedes Ergebnis wird gereviewt.
- **Kontext + präziser Prompt** schlagen jedes Modell-Upgrade.
- **Verstehen, dann ausführen** – besonders im Terminal.